



OpenAget/OAN

可信智能体互联的开放治理基础设施

身份、治理、注册、分发与可信发现

OAN 标语:

OAN 让智能体互联网从丛林法则走向基于规则的可信协作。

Jinliang Xu

China Academy of Information and Communications Technology (CAICT)

如果只记住一句话

OAN 是面向智能体资源的信任基础设施层。你可以把它理解为：把治理化资源注册、可验证身份层与可信发现网络整合在同一套体系里。

1. OAN 为开发者与运营者解决什么问题
2. 用直白语言理解 OAN 是什么
3. 为什么 OAN 不是“又一个协议”
4. 治理层、根节点、注册节点与发现节点如何协作
5. did:oan 如何支撑可移植智能体资源
6. OAN 如何补充 MCP、A2A、ANP 和 API
7. 当前已经有哪些工程证据
8. 当前性能大致处于什么水平
9. 如何试用、参与并继续建设 OAN

三个关键信息

OAN 做的是：在调用之前先建立信任、在基础设施权限之前先建立治理、在发现结果之前先提供证据。

为什么多智能体协作正在变得不可避免

多智能体互联是伪需求吗？

这里并不是把多智能体协作当成一种时髦架构选择，而是在表达一个更强的判断：当现实世界中的参与者、资源与职责逐步被智能化之后，协作会在生态层面和工程层面都变成结构性必然。

1. 宏观视角

- 知识、工具、数据源与职责，本来就分散在不同个人、组织、网站、系统和领域中。
- 当这些参与者逐步演化成软件智能体之后，原有的分布式结构不会消失，只会变成智能体之间的分布式结构。
- 在那样的世界里，协作不再是例外，而会成为生态的默认运行形态。

2. 工程视角

- 单一超级智能体最终一定会碰到协调成本、扩展瓶颈和信任域边界。
- 学术界成果表明，把工作拆分给多个智能体，有助于提升模块化、并行性、故障隔离能力和组织适配性。
- 但一旦协作跨越团队或跨越领域，就不能再依赖私下信任假设或临时交换端点。

为什么这会导向 OAN

如果多智能体协作同时是现实分布结构和工程拆分逻辑的自然终点，那么开放身份、治理、注册、分发与可信发现就会变成基础设施要求，而不是可有可无的附加项。

用直白的话理解 OAN

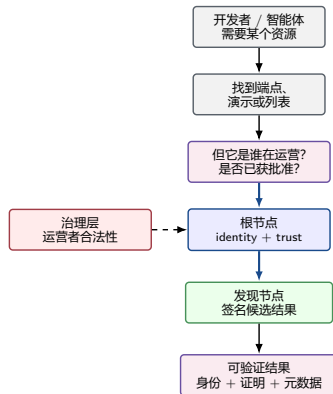
一句话解释

OAN 之于智能体资源，就像一个 **可信查号台 + 身份校验中心**：

- 它给资源提供可验证身份；
- 它让基础设施运营者在治理框架下运行；
- 它让用户在真正连接前，就拿到带签名证据的候选结果。

为什么这很重要

- 开发者需要知道 **是谁在为某个端点背书**。
- 运营者需要的是 **被治理认可的合法性**，而不只是服务在线。
- 如果结果在使用前无法 **被验证**，那么只有发现节点还远远不够。



简短版

OAN 是“我找到了一个智能体资源”和“我可以放心使用它”之间长期缺失的那一层信任基础设施。

开发者真正的痛点

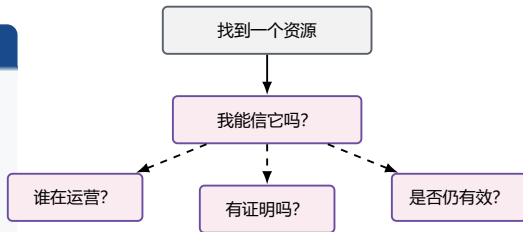
痛点是什么

在进入架构细节之前，先把问题放回开发者的日常场景。真正困难的不是“找到一个智能体资源”，而是“判断它到底值不值得信”。

典型场景

开发者想调用别人的 Agent service、Skill、MCP server 或 API：

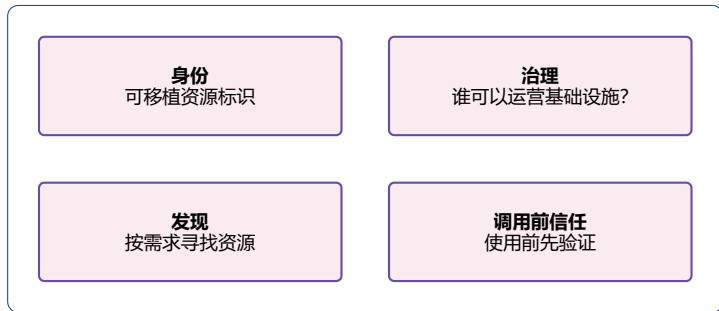
1. 他们往往可以找到端点或演示页面。
2. 但他们很难快速验证这个资源是否真实、是否仍有效、是否经过治理授权。
3. 每一次连接都像一次信任赌博：**是真的还是假的、有效还是撤销了、安全还是高风险？**



为什么会有 OAN

OAN 的意义，就是把这种信任赌博改造成一条由治理支撑、可验证、带证据的工作流。

当下生态还缺什么



- 现有协议可以连接端点，但它们并不回答：**谁是合法的、什么状态是最新的、什么对象值得信任。**
- OAN 试图补上的，正是“连通性”和“可信性”之间长期缺失的那一层。

为什么规模一上来就会变痛

在生态规模上，把“可发现”直接当成“可信”，会迅速演变成系统性风险。

来源: White paper: Identity/Governance/Discovery/Invocation gaps; identity trust layering

OAN 的核心回答

核心动作

当信任缺口被看清之后，OAN 的回答其实很简单：把开放治理、可移植资源身份与可信发现，组合成一层统一的基础设施。



定位

OAN 不只是一个注册表。它是一层面面向智能体资源、协议中立的信任基础设施：以治理为基础设施背书，以资源身份为统一入口，以带证据的发现结果为输出。

DID = 去中心化标识符；VC = 可验证凭证。

来源：White paper: Vision and Scope; yellow paper: Protocol Stack and Trust Layers

给非专业读者的小图例

快速术语表

接下来的页面会用到去中心化身份、治理和智能体互操作相关术语。先给一个简短图例，帮助读者即使不熟悉这些背景，也能顺着图表看下去。

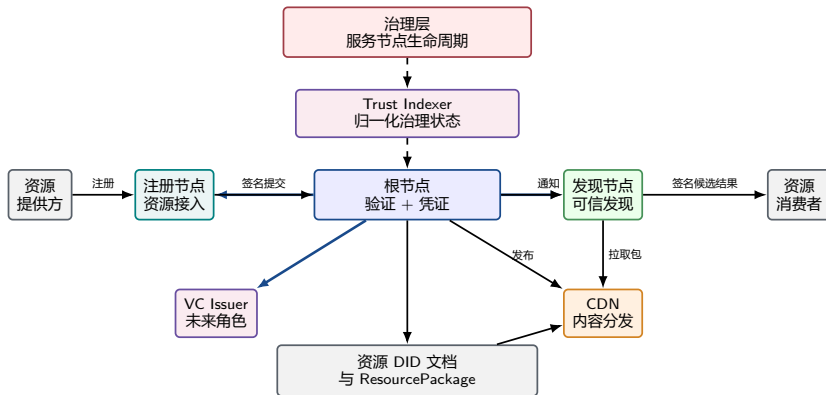
术语	通俗含义
DID	资源可移植的数字身份证。
VC	由可信方签发的许可或证明。
MCP	模型与工具交换上下文和工具的协议。
A2A	智能体之间互相通信的协议。
ANP	OAN 可以配合使用的相邻智能体网络路径。
根节点	OAN 的信任锚点，负责验证、接收与授权。

这一步为什么重要

即使读者并非去中心化身份或协议标准领域的专家，也应该能看懂后续图表。

来源: White paper terminology; DID method spec; security model

OAN 架构



治理层决定谁可以运营节点；根节点决定什么资源可以被接收和发布；发现节点决定哪些结果可以带签名证据被发现。

虚线表示观察/同步；实线表示承载信任语义的运行流。

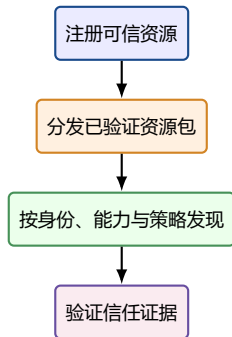
来源: System design: Roles/Trust Flow; on-chain bulletin; trust indexer design

OAN 是什么

系统边界

有了关键术语和顶层架构之后，这一页进一步把系统边界说清楚：OAN 不是单纯的标识格式，也不是另一个注册接口，而是一条完整的可信资源管道。

- 一个由治理支撑的资源身份与发现基础设施。
- 一条面向 AI 智能体相关资源的注册、发布、分发、发现与验证链路。
- 一层位于异构交互协议之上的公共信任层。



来源: White paper: What OAN Is; product model: Core Positioning

OAN 处在什么位置

在栈中的位置

当 OAN 被正面定义之后，下一步就要说明它与大家熟悉的系统之间的关系。这一页用来避免大家把 OAN 误解成通信协议，或者误解成普通应用市场。

定位	含义
位于 MCP/A2A/ANP/API 之上 聚焦可信注册与发现 强调先有证据再调用 天生开放可审查	于 OAN 不替代通信协议，而是给它们加上一层治理化身份与信任层。 OAN 不是通用应用市场，而是决定什么可以被发布、什么可以被发现的基础设施。 在真正调用端点之前，OAN 先改善身份、生命周期与发现结果的可信性。 治理状态、资源身份与发现证据都应该是可审阅的，而不是平台私有黑盒。

要点

OAN 是智能体生态下方的信任层，而不是另一个孤立的上层协议。

这套架构为什么有价值

为什么重要

说明清楚 OAN 在技术栈中的位置之后，更重要的是把这种位置转化成可感知的收益。这一页把架构选择和技术韧性、开发者收益连接起来。

技术优势

- 角色分离意味着某一个节点失陷、某一个运营者失误，不会直接决定整个网络的可信性。
- 治理可见性让生命周期和撤销状态变得可观察，而不是只能靠隐式约定。
- 根节点授权让基础设施信任成为显式决策，而不只是一个 URL 自称“我是官方”。
- 基于 DID 的身份模型让资源在协议和端点变化时仍保持可移植性。

开发者 / 生态收益

- 开发者不必再把大量时间浪费在猜测哪些资源是正规的。
- 运营者可以作为被治理的基础设施加入，而不是停留在“民间目录站”的水平。
- 发现节点可以进一步演进成按领域组织的可信生态。
- 开放规范让 OAN 更容易被审查、采用和扩展。

人话版本

OAN 帮开发者避免连错对象，也帮助整个生态避免建立在脆弱信任之上继续扩张。

OAN 的显著不同之处

维度	OAN	BUPT-AIP	CAS-AIP	ANP	AGNTCY ADS	NANDA
核心定位	信任治理基础设施 可发现资源	智能体协议体系 智能体	科研型智能体运行时 智能体 / ToolBox	去中心化智能体协议 智能体 / DID 端点	分布式智能体目录 智能体描述 / 制品	智能体索引与发现 智能体事实 / 索引
主要对象	链上 / 委员会 + Root	中心化运营方	中心化 / 网关式	社区治理	Linux Foundation / TSC / 工作组	研究联盟
治理模型	链上 / 委员会 + Root	中心化运营方	中心化 / 网关式	社区治理	Linux Foundation / TSC / 工作组	研究联盟
信任模型	分层信任 + Root 证明	审核 + 证书	mTLS / 会话信任	DID / E2EE / DNS	制品签名 + DHT	可验证事实
资源模型	智能体服务、技能、MCP、 工具/API	以智能体为中心的 ACS	智能体与 ToolBox	以智能体为中心	以智能体 / OASF 为中 心	以 AgentFacts 为中心
发现机制	授权发现; 可选 GRAIL	向量嵌入 + 过滤	非核心重点	网络 / 社区式	Kademlia DHT 目录	自适应解析
语义发现	GRAIL 可选方案	向量嵌入 + 过滤	非核心重点	较轻, 偏协议侧	目录优先	自适应发现
身份体系	did:oan	AIC / OID 风格	本地 ID / AgentInfo	DID / did:wba	身份组件 + 制品 ID	AgentFacts 标识
运行时协议	中 立; 适 配 MCP/A2A/ANP/AIP	AIP Direct RPC / Group	gRPC	ANP 协议栈	SLIM / ACP 集成	非核心重点
实现语言	Rust 核心	Python 核心	Python 核心	Python 核心	Rust 核心 / Go 组件	Python 核心
标准化	论文 + IETF 草案 + CCSA 工作	对齐国内标准	论文 + 原型	社区规范 + 草案	IETF + LF 流程	学术论文
生态基础	OpenAgenet + 试点	国内试点生态	研究 / 实验室生态	开源社区 + SDK	Linux Foundation 生态	学术网络
主要优势	连接前信任	国内试点对齐	科研型 gRPC 运行时	基于 DID 的去中心化	生态与 LF 背书	学术叙事

来源: Comparative analysis: OAN vs. BUPT-AIP, CAS-AIP, ANP, AGNTCY, NANDA

OAN 的关键差异化

它与众不同的地方

真正有价值的比较，不是 OAN 能不能也传消息，而是它补上了哪些周边系统通常说不清、或者外包出去的信任职责。这张表的目的，就是让这种差异一眼可见。

能力点	周边系统	OAN	为什么重要
连通 / 调用	✓	✓	OAN 补充的是协议连通性之上的信任层。
治理化节点授权	×	✓	OAN 明确治理谁可以运营基础设施。
先有身份再暴露资源	partial	✓	资源在被发现前就已经带有治理化身份。
发现结果自带签名证据	rare	✓	发现节点返回结果附带可验证证明。
跨域信任	limited	✓	OAN 从一开始就是为多运营者信任而设计的。

核心结论

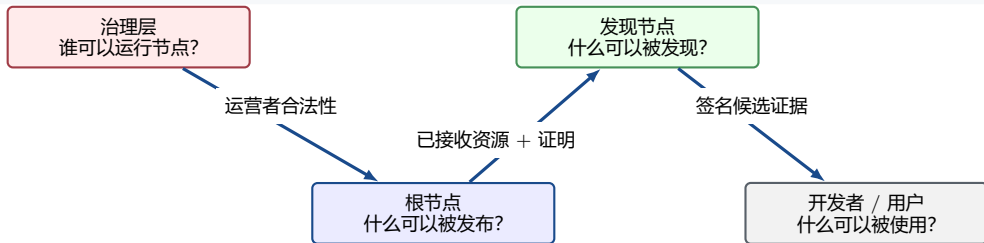
OAN 的价值不在于它也能传消息，而在于它补上了周边系统通常缺失的信任与治理能力。

来源: White paper: Relationship with MCP/A2A/ANP; yellow paper: compatibility boundaries

信任链条，一页看懂

最短的信任叙事

前面的对比页解释了 OAN 新增了什么，这一页把答案压缩成一条最短信任路径：从治理合法性出发，最终落到用户可安全选择、可验证的资源。



这一页记住什么

治理层决定谁可以运营。根节点决定什么资源被接收。发现节点决定什么结果对外可见。用户在调用前先做验证。

翻译成成人话就是：先批准运营者，再批准资源，最后只暴露可验证的发现结果。
DID = 去中心化标识符；VC = 可验证凭证；MCP = Model Context Protocol。

OAN 的差异化价值

价值总结

这一页把前面的信任链条翻译成更明确的价值表达，既总结 OAN 额外承担了哪些职责，也总结这些职责为生态带来了什么收益。

OAN 新增了什么

- 基础设施授权之前，先有治理决议。
- 资源进入目录暴露之前，先有资源身份。
- 资源发布之前，先经过根节点接收。
- 发现节点建索引之前，先有证据。
- 旧信任扩散之前，先能看到撤销状态。
- 跨平台复用之前，先有可移植的信任证明。
- 真正调用之前，先做验证。

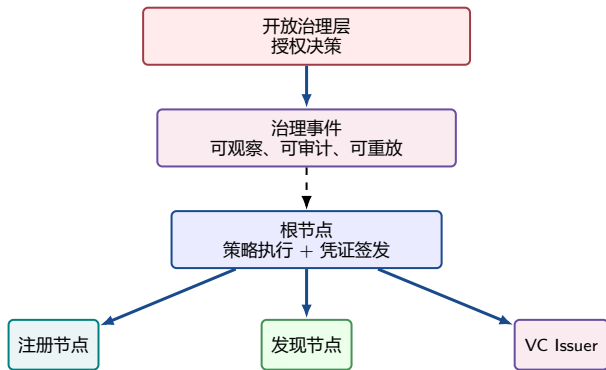
为什么这很重要

- 支持多运营者、跨域信任协作。
- 同时覆盖 Agent service、Skill、MCP server 与 Tool/API 资源。
- 在信任建立之后，原生协议可以继续各做各的。
- 让发现节点始终受授权域和可验证证据约束。
- 提供一个面向标准化的信任模型，而不是封闭平台私有规则。

定位表述

OAN 是一层协议中立的信任与发现基础设施：它不替代通信框架，而是通过治理资源身份、生命周期、发布流程和连接前验证来补全它们。

谁在控制根节点？



真正做决定的是谁

- 治理层决定哪些基础设施身份是合法的。
- 这些决定会沉淀成可重放事件，而不是运营者私下载量。
- 根节点消费这些治理状态，并把它们转化成运行时信任决策。
- 注册节点、发现节点和未来的签发者节点不会自行定义信任，它们通过根节点继承信任。
- 因此根节点很重要，但并不主权化，它的权力受开放治理证据约束。

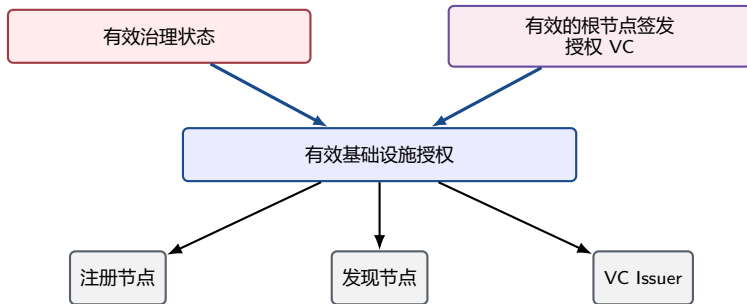
这一页的结论

根节点不是一个封闭黑盒权威，它执行的是有治理依据支撑的信任决策。

有效的服务节点授权

授权规则

授权并不是“拿到一张凭证”就结束。在 OAN 中，一个服务节点只有在其 **当前治理状态** 与 **有效的根节点签发授权** 同时成立时，才算真正被信任。



有效授权 = 有效治理状态 + 有效的根节点签发 VC

授权决策矩阵

治理状态	根节点 VC	服务结果	原因
有效	有效	接受	节点当前已获授权,并且拥有运行所需凭证。
有效	缺失/无效	拒绝依赖 VC 的服务	虽然存在治理决策,但节点尚未证明自己拥有根节点签发授权。
失效/已撤销	有效	拒绝新服务	历史 VC 不能覆盖最新治理状态。
缺失	有效或缺失	拒绝	没有治理状态,根节点无法推断该基础设施是否有权运行。

运行规则

根节点维护一份来自 Trust Indexer 的本地授权缓存,并在服务注册节点、发现节点或第三方 VC Issuer 节点之前先应用这份规则。

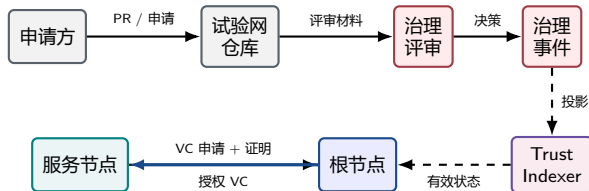
实际含义是:节点运营者不能因为“曾经拿到过一张凭证”就永久可信,当前治理状态始终优先。

来源: On-chain bulletin design; Root-indexer integration design; genesis authorization tests

服务节点准入生命周期

准入阶段

成为一个可信的 OAN 服务节点，是一个分阶段的准入流程，而不是一次“点个同意”就结束。下图展示了申请材料、治理决策、索引状态和根节点授权如何串成一条可审计的生命周期。



运行含义

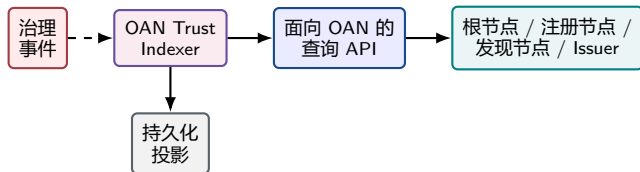
申请评审、治理表决和根节点授权是彼此分离的步骤，而且每一步都会留下可供审计的证据。

来源: On-chain bulletin: Trial-network Admission Workflow; trust indexer design

Trust Indexer: 把节点与治理运行时解耦

为什么需要 Indexer

Trust Indexer 是 OAN 与链上治理之间的适配层。它让 OAN 能够在不同智能合约链上以一致方式消费治理真相，包括联盟链和公链。



- 读取治理事件。
- 重建当前主体状态。
- 暴露稳定的 OAN 信任 API。
- 把异构链适配成统一的 OAN 信任视图。
- 支持联盟链和公链治理后端。
- 让服务节点远离原始治理运行时复杂度。

设计规则

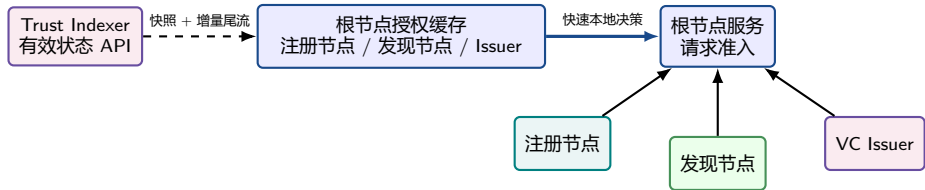
服务节点消费的是面向 OAN 的信任状态；事件翻页、重放、投影和游标管理这些复杂性都由 indexer 吸收。

来源: Trust indexer design: Position, Goals, Normalized Domain Model

根节点运行时执行

运行时决策路径

运行时信任执行既要正确，也要足够快。根节点不会在每个请求上都直接查询原始治理基础设施，而是持续吸收已索引的信任状态，并把它转化成本地准入决策，从而及时接收有效节点、切断已撤销节点。



- 根节点不会在每一次服务交互时都直接查询原始治理运行时。
- 它会同步治理状态、缓存有效服务节点权限，并停止为被撤销节点提供服务。

来源: Root-indexer integration design; Root service admission logic

治理状态与授权 VC 的边界

边界澄清

一个常见困惑点，是治理状态和凭证之间的边界。这一页说明：如果当前治理状态已经不再支持信任，那么仅靠一张历史授权 VC 是不够的。

对象	它证明了什么	它不能证明什么
治理状态	该基础设施主体在治理体系下是有效、暂停还是已撤销。	它不是协议凭证，也不能单独证明密钥控制权。
根节点签发授权 VC	根节点曾向某个 DID 主体签发过运行授权凭证。	如果最新治理状态已失效、缺失、过期或撤销，它本身并不充分。
主体控制证明	申请方控制着本次请求所使用的 DID 密钥。	它不能替代治理批准，也不能替代根节点授权。
签名请求封套	请求体、路径、方法、受众、nonce 和时间戳都被签名。	它不能赋予一个未经批准的基础设施节点以授权。

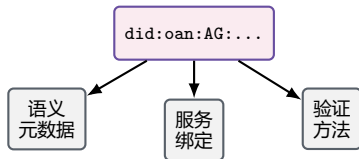
来源: Identity trust layering; on-chain bulletin: Architectural Position; oan-service-security

为什么是 did: oan

为什么选择这个标识符

OAN 需要一种在资源细节持续演进时仍保持稳定的标识符。这一页解释为什么 `did: oan` 能作为锚点, 把身份、元数据、服务绑定与证明一起稳定地串起来, 而不是塌缩成只和某个端点绑定的命名。

- 面向 AI 智能体相关资源的可移植标识符。
- DID 文档承载验证方法与服务绑定。
- 语义元数据支持按用户需求进行发现。
- 独立于具体调用协议。



规范文档: [did: oan Method Specification.md](#)

设计意图

`did: oan` 要解决的, 是在端点、语义元数据、资源包版本和协议绑定不断演进时, 资源标识本身依然稳定。

一个 DID 方法，四种产品形态

一种方法，多种形态

OAN 不会为每一种资源类型都重新发明一套身份模型。它使用同一个 DID 方法支撑多种产品形态，同时保持注册、信任证明和发现语义的一致性。

产品形态	代码	发现语义
Agent Service	AG	可调用的智能体服务，包含端点与协议绑定。
Skill	SK	能力描述，包含制品引用和实现链接。
MCP Server	MC	暴露工具、资源或提示词的 MCP 兼容服务器。
Tool/API	TL	可调用的 API、连接器、工作流或工具接口。

共享模型

这四种形态都共享 `did: oan`、DID 文档元数据、ResourcePackage、Root proof 与发现节点索引。

来源: Product model: Resource Types; oan-core ResourceType subject-code mapping

资源形态：注册的是什么，发现的又是什么

不同类型，变化在哪里

当统一 DID 方法建立起来后，下一个问题就是：不同资源形态在注册与发现里具体长什么样？这一页在不破坏共享信任模型的前提下，对这些形态做并列比较。

形态	被注册的 DID 文档承载内容	发现节点返回什么	消费者下一步
Agent Service	端点、协议绑定、语义画像、密钥、VC 提示等。	候选服务 DID 文档及其签名证据。	先验证证据，再按支持的协议调用。
Skill	语义描述、制品 URI/哈希、示例、兼容性元数据等。	候选 Skill DID 文档及制品引用。	到引用位置拉取制品并验证哈希。
MCP Server	MCP 端点、能力摘要、resource/tool/prompt 元数据。	候选 MCP Server DID 文档。	先验证，再通过 MCP 建连。
Tool/API	端点或 OpenAPI 引用、认证提示、操作语义等。	候选 API / Tool DID 文档。	先验证，再按 API 契约调用。

统一点在哪里

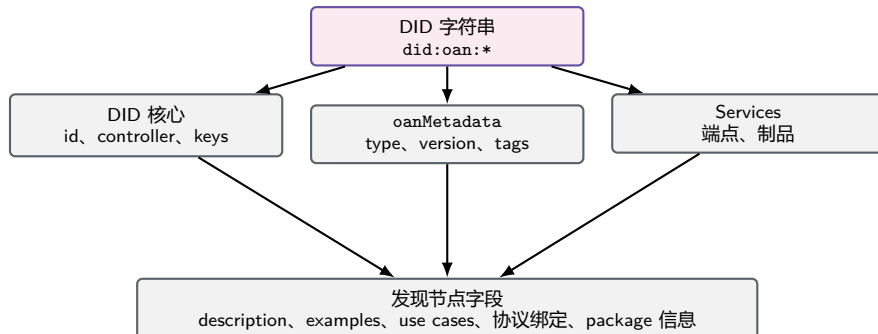
资源类型改变的是预期元数据结构，但注册、根节点验证、包证明以及发现节点证据链仍然是共享的。

来源: Product model: four product forms; DID method spec: oanMetadata and services

DID 文档就是发现面

为什么 DID 文档很关键

看完不同资源形态之后，这一页聚焦它们共享的那个公共表面。DID 文档不只是一个标识符，它还是发现过程中的结构化入口。



为什么这件事重要

在 OAN 里，发现节点索引的不是某种临时拼出来的注册记录，而是资源 DID 文档的结构化公共表面。

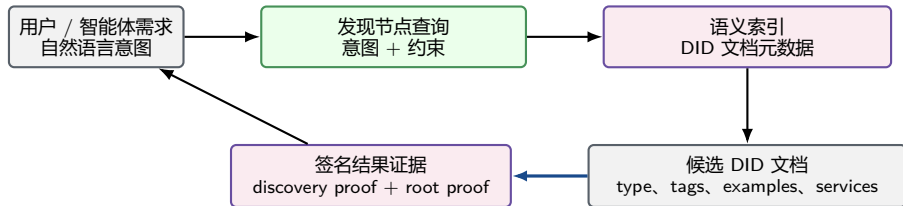
这也是为什么身份、语义、协议绑定和 package 引用能够天然连在一起，而不是散落在互不相关的记录中。

来源: DID method spec; oan-core OanMetadata; product model Discovery Flow

语义发现建立在 DID 文档之上

语义到底放在哪里

OAN 的语义发现，是从结构化 DID 文档元数据出发，而不是从临时目录行出发。正因如此，语义、身份与信任证据才能在整条发现链路里保持一致。



重要边界

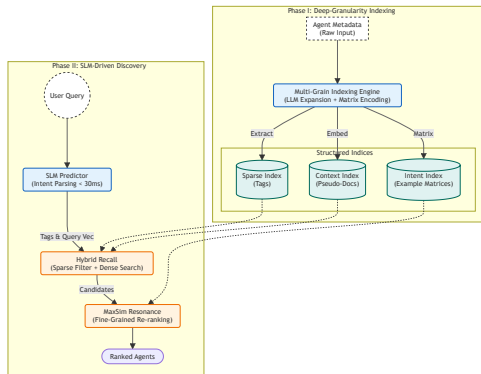
发现节点负责索引并返回带证据的 DID 文档；它不需要亲自托管资源引用到的每一个外部制品。

来源: GRAIL paper for semantic discovery direction; product model: discovery boundary

GRAIL: 发现层的语义搜索引擎

从语义元数据到语义检索

当 OAN 把语义信息纳入 DID 文档表面之后，下一个问题就变成：发现节点该如何在规模化条件下高质量地搜索这层语义表面。GRAIL 是发现层上的算法答案：它是一套面向真实生态负载而设计的专用语义发现引擎，目标是在保持高精度的同时实现快速的智能体 / 资源检索。



GRAIL 带来了什么

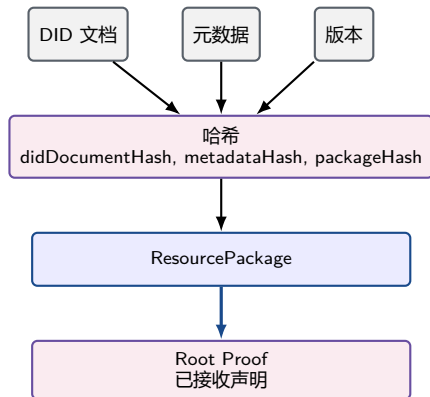
- **SLM 增强预测**: 用快速的能力标签预测替代沉重的在线 LLM 解析。
- **三维索引**: tags、context 和 intent 分开建模，而不是压缩成一个单体向量。
- **MaxSim resonance**: 让用户意图与离散 usage examples 做匹配，减轻语义稀释问题。
- **实时目标**: 框架以亚 400ms 级发现为目标，同时不牺牲检索质量。

为什么它属于 OAN

这很重要，因为 OAN 不只是重视身份和信任证明，也在发现节点的语义搜索侧投入了很深的研究和工程工作。于是，发现节点背后既有治理约束下的信任证据链，也有一套严肃的语义检索架构作为支撑。

资源包与证明各自扮演什么角色

当 DID 文档及其元数据被接收之后，OAN 还需要一个可发布对象，以及一个可移植的信任声明。这一页说明 package 和 Root proof 如何协同提供这两样东西。



package 做了什么

ResourcePackage 把一个提交上来的资源转换成一个带版本、带哈希绑定的发布单元，成为 Root 可以正式背书的对象。

Root proof 做了什么

Root proof 是面向下游节点与消费者的信任桥梁：它让他们能够验证 DID 文档、元数据和 package 版本，确实与根节点真正接收的内容一致。

为什么两者都需要

package 负责组织可发布对象，proof 负责把它变成可验证证据，供 CDN、发现节点和最终用户使用。

ResourcePackage 绑定规则

到底绑定了哪些东西

上一页介绍了 package 和 proof，这一页进一步明确它们到底绑定了哪些字段。正是这些绑定关系，防止下游验证结果偏离根节点实际接收的事实。

绑定项	这一页中的含义
resourceDid	稳定的资源主体标识，而不是会变化的端点或版本字符串。
resourceType	必须与 DID subject code 和 DID 文档中的 oanMetadata 一致。
packageVersion	该资源经 Root 验证后的版本化发布状态。
didDocumentHash	把提交的 DID 文档绑定到 Root 接收事实。
metadataHash	绑定语义与产品元数据。
packageHash	绑定 package 制品本身与其版本化发布单元。
rootProof	把 Root 接收事实转化为下游可验证证据。

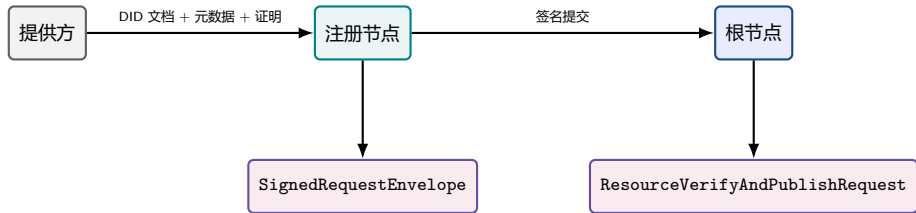
为什么这些绑定很重要

消费者不需要盲目信任发现节点；他们可以自己验证返回的文档和元数据，是否真的匹配根节点接收过的 package 事实。

资源注册时序

注册如何变成可信发布

这里是资源提供方的对象真正进入 OAN 治理管道的地方。这个时序图突出展示了注册节点如何转发带签名的提交，以及根节点如何在接收发布前同时检查权限和内容完整性。

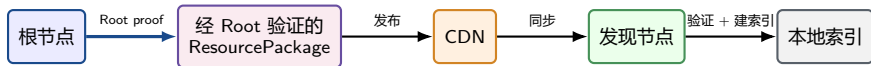


根节点会验证注册节点授权、哈希、类型一致性、新鲜度、nonce 以及主体控制证明。

来源: System design Trust Flow; oan-protocol ResourceVerifyAndPublishRequest; registrar/root code

根节点接收之后发生什么

根节点接收并不是生命周期的终点。下一步是把经验证的 package 分发到 CDN 和发现节点，同时不中断下游节点与消费者所依赖的那条证据链。



- CDN 负责分发经根节点批准的 package，但它本身不是信任权威。
- 发现节点负责索引已验证 package 以及 DID 文档元数据。

这条管道的边界

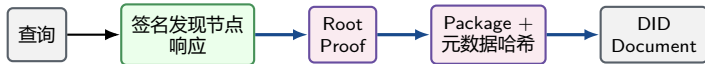
发布可以是异步的，但每一份下游副本都必须能够追溯到同一份 Root proof、package hash 和 DID 文档 hash。

这就是核心承诺：异步分发，但不丢失任何一个信任锚点。

来源: System design route graph; root repository queues; CDN and Discovery service code

证据路径

OAN 中的发现节点信任是累积出来的。一个返回候选之所以有意义，是因为这条响应可以沿着 Root proof、哈希和 DID 文档事实一路向后追溯到可验证对象。



带证据的发现

发现节点返回的不是“你先信我”，而是可以追溯回根节点已验证资源事实的候选结果。

翻译成成人话：发现节点不是在说“相信我”，而是在说“证据链在这，你自己验证”。

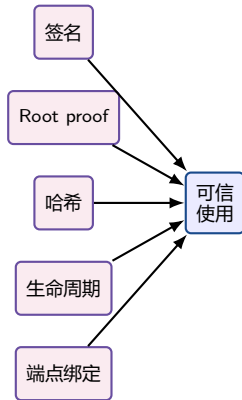
来源: Yellow paper: Discovery Response and verification algorithms; oan-protocol ResourceDiscoveryResponse

消费者到底验证什么

验证清单

OAN 不要求消费者盲目信任搜索结果，而是提供一份具体的验证清单，把发现证据转换成“这个资源是否足够安全、足够新鲜、值得使用”的理性判断。

- Discovery 响应签名。
- Root proof 及已接收声明。
- DID 文档 hash 与 metadata hash。
- package hash 及哈希算法。
- 资源类型、生命周期状态和版本。
- 端点或制品绑定关系。



发现节点查询契约

查询边界

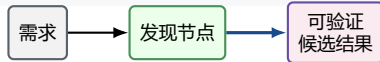
看完资源如何被发布、发现和验证之后，有必要把实际查询边界总结出来。这一页把发现节点描述为“用户意图”和“可验证候选证据”之间的契约。

查询输入

- 需求描述与关键词
- 可选的 resourceType
- 能力标签与自定义标签
- 协议绑定或端点偏好
- 版本或生命周期约束

响应证据

- 候选 resourceDid
- 类型、版本、生命周期状态
- 能力标签与服务绑定
- package 信息与 Root proof 引用
- 发现节点响应证明



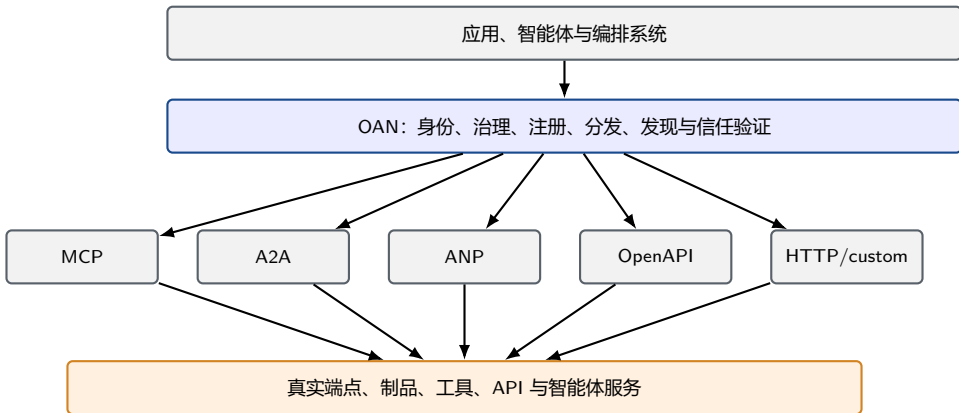
为什么这有利于落地

开发者在发现节点之后仍然可以继续使用自己熟悉的协议。OAN 改变的是候选对象如何被识别、如何被信任，而不是强迫所有下游调用都重写。

OAN 位于异构协议之上

兼容性边界

只有当信任与发现管道被说清楚之后，协议兼容性的位置才容易被看明白。这一页说明：OAN 位于异构调用协议之上，而不是试图替代它们。



来源：White paper: relationship with MCP/A2A/ANP; yellow paper: compatibility boundaries

MCP Server 与 Tool/API 映射

两种资源 profile

即使共享同一条信任管道，不同资源形态也需要不同的元数据表面。这一页说明 OAN 分别期待 MCP Server 与 Tool/API 提供哪些信息，才能保持可发现、可验证。

MCP Server

- did:oan 资源类型: mcp_server。
- 描述端点、传输方式与协议版本。
- 概述 tools、resources 与 prompts。

Tool/API

- did:oan 资源类型: tool_api。
- 链接 API schema 或 OpenAPI 描述。
- 描述认证方式、端点、示例与校验信息。

具体例子：注册一个 MCP Server

一个具体 walkthrough

抽象映射关系通过一个具体案例会更容易理解。这个例子展示了一个熟悉的 MCP server 如何从注册开始，沿着 OAN 信任管道一路走到可验证使用。

1. 准备一个描述 MCP server 端点、能力、验证方法与元数据的 DID 文档。
2. 通过注册节点携带签名请求证据提交它。
3. 根节点验证提交内容，并把它接入治理化发布路径。
4. 发现节点之后会把这个 MCP server 作为带证据的签名候选结果返回。
5. 开发者先验证证据，再通过 MCP 建连。

OAN 改变了什么

没有 OAN 时，开发者看到的只是一个 MCP 端点；有了 OAN 之后，开发者在连接前就能看到治理化身份、根节点接收事实和签名发现证据。

开发者真正获得了什么

更少时间花在猜测这个 MCP server 是不是真的、是不是最新的、是否值得接入；更多时间可以用在真正构建能力上，因为候选对象已经自带可验证信任上下文。

人话版：OAN 帮你判断这个 MCP server 到底是正规服务，还是互联网上一个随机端点。

来源：Product model: MCP Server flow; Root publication path; discovery evidence path

Agent Service 与 Skill 映射

两个不同角色

Agent service 和 skill 经常在产品讨论中一起出现，但它们在模型里扮演的是不同角色：一个是可调用的运行时端点，另一个是可复用的能力描述或制品引用。

Agent Service

- 可调用的业务智能体。
- 端点与协议绑定。
- 可暴露 A2A、MCP、HTTP 或自定义协议。

Skill

- 能力描述。
- 制品 URL、哈希与版本。
- 可链接到 Agent Service、MCP Server 或 Tool/API 实现。

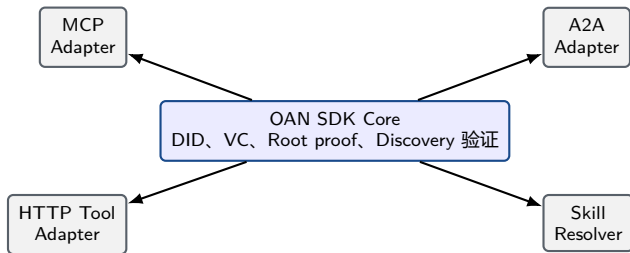
边界

发现节点索引的是 DID 文档元数据与引用关系，而默认并不亲自托管 Skill 文件。

Adapter 与 SDK 路径

采用路径

当信任逻辑被集中沉淀，而不是在每个集成场景里重写一遍时，采用成本就会显著下降。OAN 的 SDK core 提供共享验证模型，而 adapter 则把它翻译成面向具体协议的开发流程。



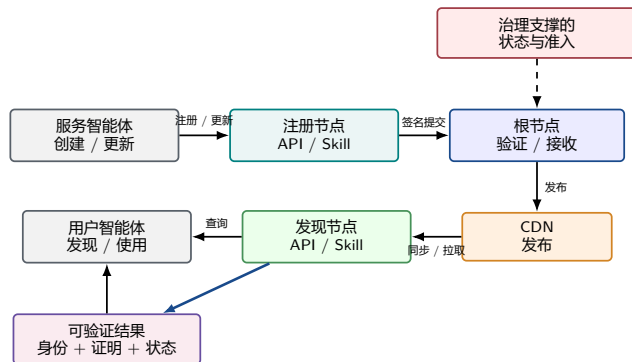
目标

开发者不应该手搓信任验证逻辑；SDK 和 adapter 应该把“默认安全使用”变成常态。

OAN 是 AI 原生基础设施

从开发者工具走向智能体到智能体的基础设施

SDK 只是近期采用路径的一部分。更深层的设计点在于，OAN 暴露的是可被机器直接消费的信任交互面，因此智能体可以直接通过 OAN 完成注册、更新、发现、验证与互联，而不必依赖人工去驱动每一步。



这一页在表达什么

- 智能体资源可以通过 Skill 或 API 被直接注册和更新。
- 发现节点可以返回带签名的候选结果，供另一个智能体自动验证。
- 身份、package 和 proof 是协议对象，而不只是后台页面里的记录。
- 治理支撑的状态可以被软件直接消费，并进入自动化信任决策流程。

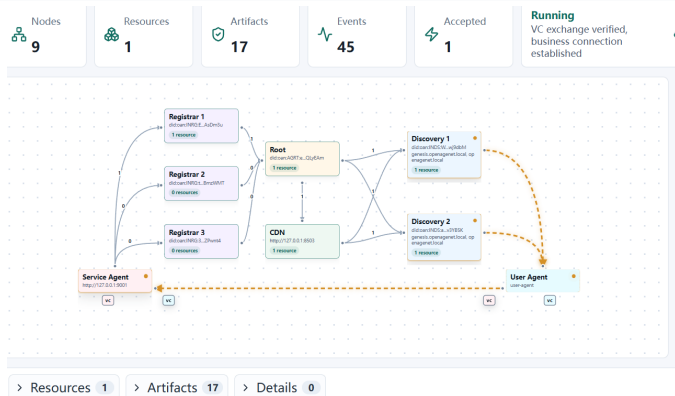
为什么这很重要

OAN 不只是一个供人浏览的目录，而是一层可被智能体直接使用的信任基础设施，使它们能够彼此创建、注册、发现、验证并建立互联。

Demo 视图：单个 Agent 端到端

最小但完整的信任闭环

一个 service agent 沿着完整 OAN 路径走到一个 user agent，在引入规模之前，先让最完整的信任闭环变得一眼可读。



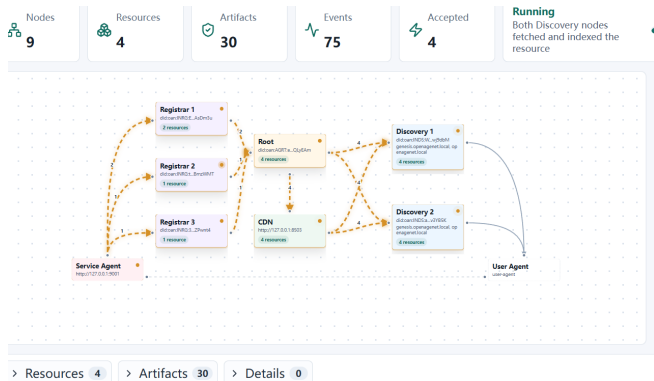
最小闭环：注册节点提交，根节点接收，CDN 发布，发现节点建索引，业务连接只会在信任验证之后发生。

来源: oan-demos scenario screenshot: one-agent

Demo 视图：四种混合资源

同一拓扑，多种资源形态

在 one-agent 基线之后，这一页展示了不同资源形态如何共享同一条信任发布管道。



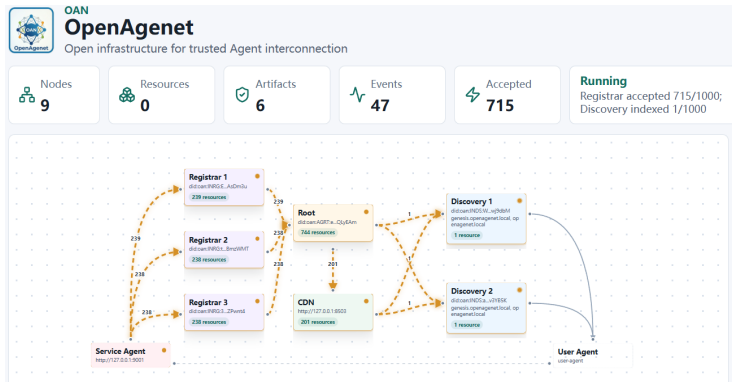
三个注册节点提交资源，根节点聚合并接收它们，CDN 再分发，两台发现节点索引的是同一组受治理约束的资源。

来源: oan-demos scenario screenshot: four-mixed

Demo 视图：1000 个混合资源

从正确性走向运行压力

这个大规模视图把压测叙事变成了直观画面：在更高负载下，整条管道依然在工作，发布与发现也在持续推进。



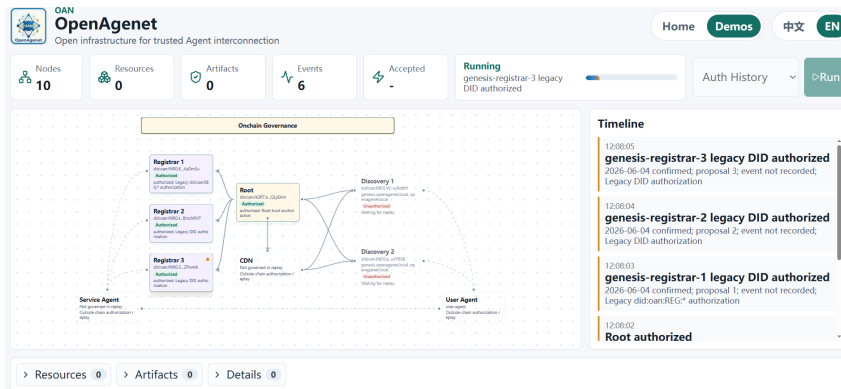
计数器揭示了关键运行事实：根节点接收最先推进，发布随后跟上，发现节点的可见性再沿着流逐步追平。

来源: oan-demos scenario screenshot: 1000-mixed

Demo 视图：授权历史

治理是可见的，而不是隐藏的

前面的页面强调的是发布和发现，这一页则直接把治理侧呈现成一条可读的授权时间线。



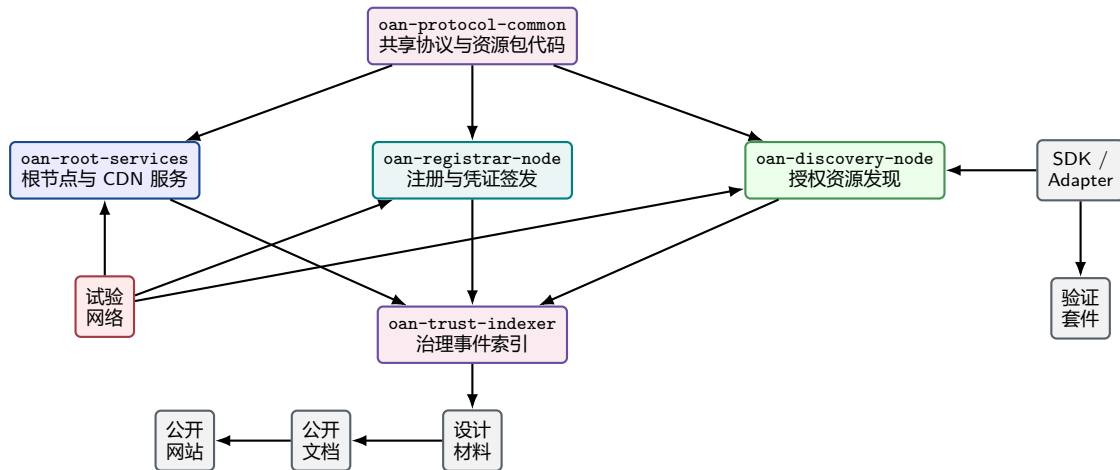
读者不仅能看到节点是否已授权，还能看到它们是在何时、基于什么原因获得授权的。

来源: oan-demos scenario screenshot: auth-history

多仓工程栈

仓级视图

在讲清协议和生命周期之后，这部分内容从架构转向工程组织方式。本页说明实现如何分布在多个仓库中，同时又不把整套信任模型拆散。



来源: System design: split node repositories and shared protocol libraries

节点服务与共享对象

代码边界

实现边界之所以重要，是因为 OAN 并不只是一个二进制程序。本页把运行中的节点角色和共享协议 crate 区分开，说明系统如何在保持信任逻辑统一的同时维持模块化。

Rust 节点服务

- 根节点
- 注册节点
- 发现节点
- CDN 服务

共享协议 Crates

- oan-core, oan-did-oan
- oan-protocol, oan-package
- oan-crypto, oan-credentials
- oan-storage, oan-client

- 运行态状态以 PostgreSQL 为主；SQLite 仍适合本地开发和测试。

来源: System design: Implementation Stack; database abstraction analysis; protocol common crates

当前正式 API 面

集成边界

当实现模块已经可见之后，下一个实际问题就是接入方到底该从哪里集成。本页总结清理后的 API 面，它比旧的路由布局更清楚地暴露了系统边界。

角色	路由前缀	用途
根节点	/root/...	验证、发布、bulletin、授权与队列操作。
注册节点	/registrar/..., /resources/...	资源接入、注册提交、能力树访问。
发现节点	/discovery/...	资源同步、查询、路由查找、带签名的发现响应。
CDN	/cdn/resources/..., /cdn/documents/..., /cdn/metadata/...	分发经 Root 验证的资源包，以及文档/元数据视图。

当前边界

当前正式 API 面已经不再在主服务路径中保留遗留的 `/api/v1/...` 路由。

这对接入方很重要，因为系统边界正在变得更干净、更容易理解，也更不容易误导后续的集成与重构工作。

来源: System design: Formal Implementation Boundary; service main.rs route definitions

文档与规范资产

文档分层

OAN 有意不只是一套代码。本页展示内部文档分层，它帮助项目对工程师、评审者、合作伙伴以及未来面向标准化的受众都保持可解释性。

资产	作用
White Paper	愿景、价值、治理与生态定位。
Yellow Paper	技术架构、协议对象、生命周期与安全。
DID Method Spec	did:oan 的公开规范界面。
Detailed Design Docs	系统、治理、资源模型、索引器、数据库、网站等详细设计。
Public Docs	面向社区的论文、规范和精选 PDF。
Slides	面向领导、合作伙伴、开发者和标准化交流的沟通材料。

要点

OAN 正在被构建为“代码 + 规范 + 治理材料”的组合体，而不只是一个服务演示。

来源: [whitepaper](#)/[yellowpaper](#); [did-oan spec](#); [docs/system](#); [oan-public-docs](#)

公开论文与标准化产出

对外产出

这些材料合在一起，使 OAN 不只是代码仓库，也是一项工程工作、一组研究产出，以及一个可进入标准化讨论的候选架构。

白皮书 / 黄皮书

OpenAgenet/OAN White Paper:
Open Infrastructure for Trusted Agent
Interconnection [arXiv:2606.03161](#)

OpenAgenet/OAN Yellow Paper:
Technical Architecture for Trust-Governed
Resource Identity and Discovery
[arXiv:2606.03163](#)

Slides: OpenAgenet/OAN Open
Governance Infrastructure for Trusted
Agent Interconnection [GitHub repo](#)

学术论文

Trusted Management Model for the Lifecycle of
Agent Identity Documents

AONA: A Comprehensive Architecture and
Workflow Design for Global Agentic Collaboration

GRAIL: A Deep-Granularity Hybrid Resonance
Framework for Real-Time Agent Discovery via
SLM-Enhanced Indexing [arXiv:2605.02489](#)

DarwinNet: An Evolutionary Network
Architecture for Agent-Driven Protocol Synthesis
[arXiv:2604.01236](#)

IETF 标准草案

**draft-xu-oan-resource-identity-
discovery-00:** [Trust-Governed
Resource Identity and Discovery
Architecture for OpenAgenet](#)

**draft-xu-agentic-overlay-network-
architecture-00:** [Architecture for
Agentic Overlay Networks](#)

**draft-xu-efficient-agent-discovery-
profile-00:** [Metadata and Query
Profile for Efficient Agent
Discovery](#)

要点

OAN 正在以开源工程项目、公开研究产出和面向标准化的架构提案这三种身份同步推进。

叙事背后的工程证据

从主张到代码的对应关系

本页把项目的对外主张与具体工程证据对应起来，使整套叙事保持可检查、可追溯。

主张	工程证据	为什么重要
统一资源身份	oan-core, oan-did-oan, DID method spec	身份规则在服务、文档和测试之间共享。
受治理约束的服务节点	治理设计、Trust Indexer、genesis 身份材料	基础设施授权是可观察、可测试的。
经根节点验证的发布链路	oan-package、根节点服务、CDN 服务	发现节点结果可以追溯回根节点的接收与确认。
多资源类型支持	Agent、Skill、MCP、Tool/API 的验证用例与协议模型	OAN 并不局限于单一 Agent 服务形态。
开发者接入路径	SDK、Adapter、公开文档与网站材料	生态参与者能拿到实际可用的接入材料。

证据模式

每一项主要主张都同时有设计材料和实现证据支撑，使项目不是停留在概念层，而是可被审视的工程系统。

换句话说，这不只是一个关于未来信任基础设施的故事，它已经是一套可以跑、可以测、可以核验的工程系统。

来源: Core protocol libraries; node service implementations; validation material; public documentation

安全与信任属性

OAN 保护了什么

到这里，关键问题已经不只是 OAN 如何工作，而是它实际保护了什么。下面两栏把基础设施信任与资源信任分开，帮助读者同时看见它提供的保证，以及它有意保留的边界。

基础设施信任

- 治理状态可审计；
- 根节点授权 VC 显式存在；
- 服务节点请求带签名；
- 被撤销节点会被根节点缓存拒绝。

资源信任

- DID 文档绑定密钥与服务；
- ResourcePackage 绑定哈希与版本；
- Root proof 绑定被接收的声明；
- 发现节点响应携带可验证证据。

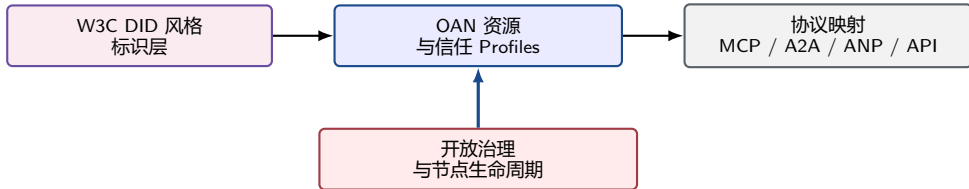
作用范围

OAN 强化的是身份、生命周期与发现链路的信任；它并不替代业务级安全、模型评估，或端点自身的安全控制。

面向标准化的定位

标准化视角

OAN 不只是工程栈，它也是一套可以用标准化语言来描述的候选架构。本页把它的 DID 层、信任 profile、治理模型与协议映射连接到更大的图景中。



- OAN 可以被表述为面向智能体生态的资源身份与可信发现 profile。
- 随着模型逐步稳定，项目既适合先做开源协作，也适合进入面向标准化的讨论。

来源: DID method spec; public docs; white/yellow paper standards discussion

性能快照：当前系统已经证明了什么

基线已经说明了什么

讲完架构和实现之后，下一个问题就是系统是否已经在有意义的规模上跑通。本页先总结当前基线证据，再把叙事引向剩余瓶颈。

运行结论

- 当前实现已经能完成 **1000 资源**规模的可信发布与发现流程，且 **missing = 0**。
- 单节点混合资源流水线在 1000 资源规模上达到约 **57.22 resources/s**。
- 多节点混合资源流水线在 1000 资源规模上达到约 **47.90 resources/s**。
- 剩余挑战已经不再是正确性，而是吞吐量与尾延迟。

四类压测场景怎么区分

- **Pipeline**: 单节点 + 单一资源类型。
- **Mixed-resource**: 单节点 + 混合资源形态。
- **Multi-node pipeline**: 多节点 + 单一资源类型。
- **Multi-node mixed**: 多节点 + 混合资源形态。

两条对比轴

- **拓扑轴**: 单节点部署 vs. 多节点部署。
- **资源轴**: 单一资源形态 vs. 异构资源混合。

一句话读懂

当前的问题已经不再是“OAN 能不能工作”，而是“OAN 在保持治理约束与可验证发现的前提下，究竟还能扩展到多大规模”。

如何阅读压测结果：系统强在哪里，又卡在哪里

怎么看这些结果

压测快照只有在能导向诚实解读时才有意义。本页把“当前数据已经证明了什么”和“下一轮优化仍需聚焦哪里”拆开来说。

维度	当前数据已经说明了什么	仍需继续优化的地方
正确性	已完成的基线运行都能以 <code>missing = 0</code> 结束。	仍需要更多面向运营规模和长时间运行的证据。
资源覆盖	Agent Service、Skill、MCP Server 和 Tool/API 都走同一套信任流水线。	混合资源场景下的尾部时延仍偏慢。
单节点路径	中等规模吞吐已经足以支持真实工程验证。	大规模运行时的可见性延迟仍然上升明显。
多节点路径	OAN 已经能在同一信任模型下同时跑根节点 + 多注册节点 + 多发现节点。	根节点发布尾部和发现节点可见性仍是主要瓶颈。

给读者的重要信息

这些压测表格是工程证据，不是营销幻象。它们证明 OAN 已经是一条真实可跑的信任流水线，同时也准确暴露出下一轮优化应该落点在哪里。

来源: Performance and scalability analysis; PostgreSQL bottleneck analysis

参与路径

OAN 的参与方式有意不只限于代码贡献。发布资源、运行受治理约束的基础设施、构建 adapter，或审阅公开规范，都可以增强这个生态。



- 发布 Agent Service、Skill、MCP Server 或 Tool/API 资源。
- 申请运行注册节点、发现节点或未来的 VC Issuer 节点。
- 贡献 adapter、示例、文档与一致性测试。

来源: On-chain bulletin admission workflow; product model Registration/Discovery Flow; validation material

接下来往哪里走

下一阶段的目标，是把 OAN 推进为一条可信、国际化的智能体互联网信任基础设施路径：治理可靠、协议中立、运行成熟，并且能够被全球范围清晰理解。

阶段推进： 奠基 → 扩展 → 全球规模的信任基础设施

信任治理内核

奠基： 身份、Root proof、可防重放授权
扩展： 一致性、凭证生命周期、治理重放
规模化： 联邦域、跨域信任交接

协议中立的互操作

奠基： 面向 MCP、A2A、ANP 和 API 的 adapters
扩展： schema 映射、能力 profiles、更丰富的 SDK
规模化： 面向异构 Agent 协议的默认信任层

运行成熟度

奠基： 性能、可观测性、可复现实部署
扩展： 合作方节点、诊断工具、验证套件
规模化： 联盟级与电信级多运营方可信运行

生态与标准化影响力

奠基： 论文、Demo、公开规范
扩展： 面向研究界、产业界与标准社区的试点
规模化： 成为全球可理解的受治理约束基础设施选项

总体方向： 不是再造一个单体化 Agent 协议，而是为大规模、更安全的互联提供信任与治理底座。

来源：Agent interoperability comparison analysis; White paper roadmap; yellow paper Open Technical Work



OAN 正在帮助智能体互联网从“野蛮生长式信任”走向 **可验证的信任基础设施**。

AI 智能体生态需要的不只是协议，还需要规则、证据与开放的信任基础设施。

发布
资源

运行
服务节点

构建
SDK 与 Adapter

审阅
规范

下一步

试试代码、看看规范、跑跑 Demo，一起塑造面向智能体生态的开放信任层。

一句可转发的话

OAN：让智能体网络从“我找到了一个端点”走向“我可以先验证、再信任”的信任层。

<https://github.com/OpenAgenet>

xujinliang@caict.ac.cn; jlxfly@gmail.com

来源: White paper conclusion and stakeholder value; public docs